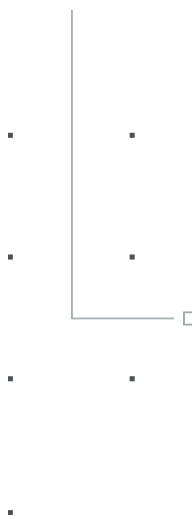


FIAP

NBA



Support Vector Machines e Regressão Linear

Dheny R. Fernandes

1. SVM

1. Intuição
2. Conceitos Básicos
3. Kernels:
 1. Polinomial
 2. Kernel trick
 3. RBF

2. Regressão Linear

1. Hipótese
2. Função de Custo
3. Gradiente Descendente

3. Regressão Linear Multivariada

1. Hipótese e Gradiente Descendente
2. Usando Scikit Learning

4. Calculando Parâmetros Analiticamente

5. Métricas de Avaliação

1. Matriz de confusão
2. Acurácia
3. Precision
4. Recall
5. F1-Score

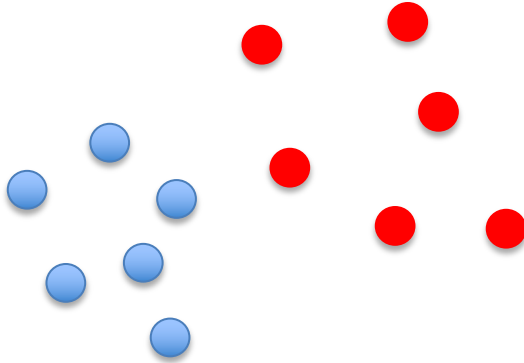
Support Vector Machines - SVM

Apresentado em 1979 por Vladimir Vapnik, é um dos algoritmos de classificação (apesar de haver sua variante para regressão) mais sólidos já criado.

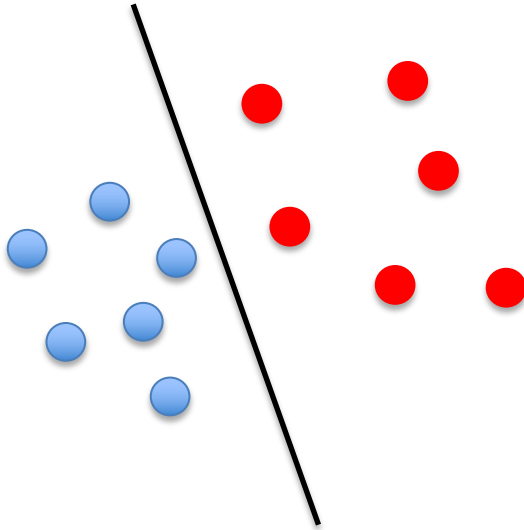
Com a existência de duas ou mais classes rotuladas de dados, ele age como um classificador discriminativo, formalmente definido por um hiperplano ideal que separa todas as classes.

Novos exemplos são classificados de acordo com a posição no espaço em que se encontram.

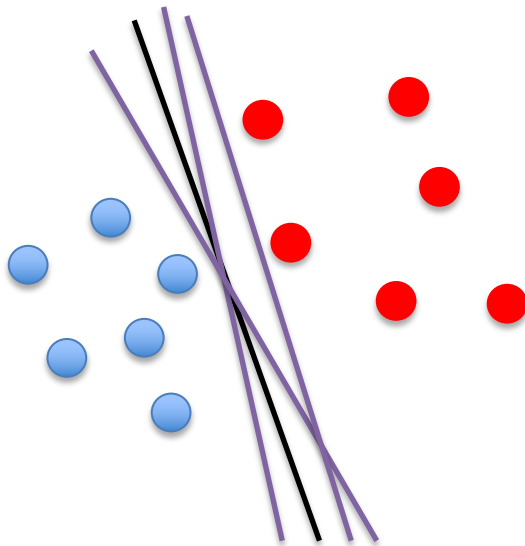
Suponha o seguinte conjunto de dados



A seguinte reta é capaz de separar os elementos de cada classe



Mas essas também são:



A questão é como encontrar uma
reta boa o suficiente de modo o
algoritmo generalize bem para
amostras ainda não vistas

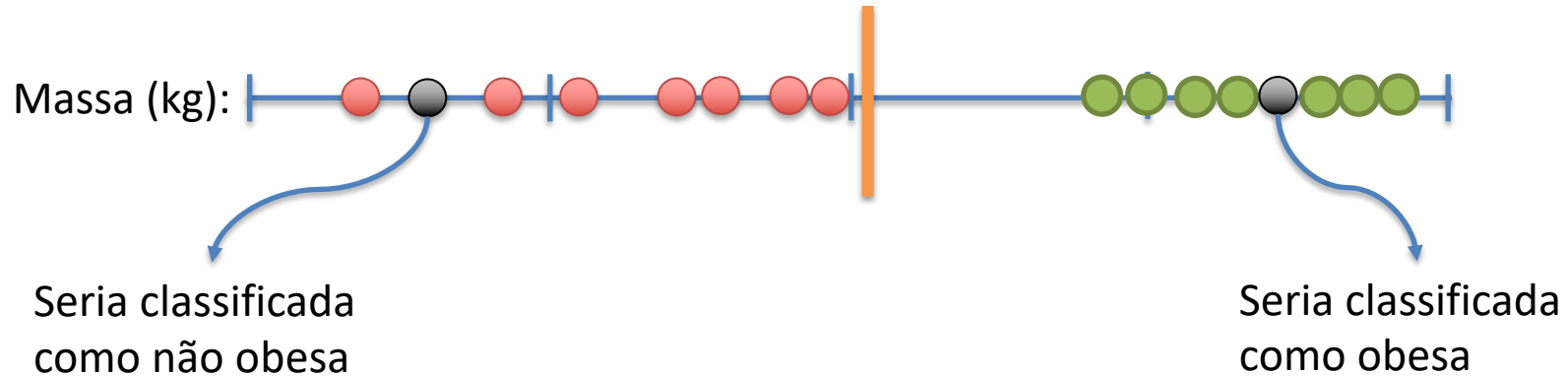
Vamos supor que mensuramos a massa de alguns cachorros:



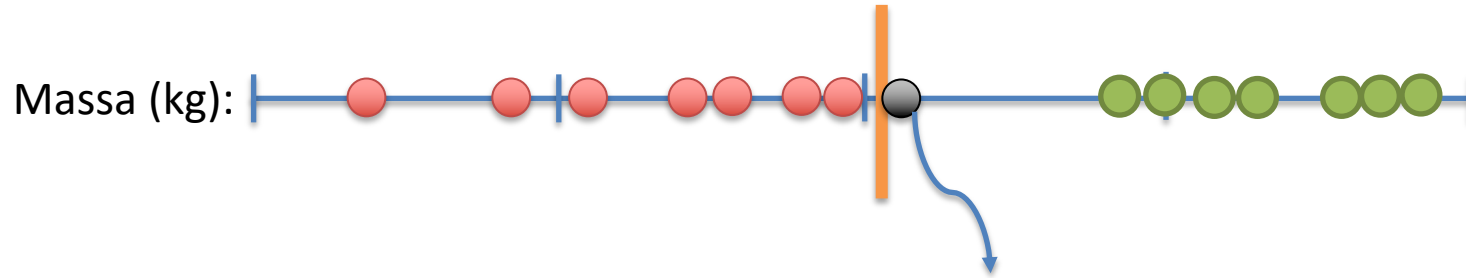
Os pontos em **vermelho** representam aqueles que não são obesos, enquanto os pontos em **verde** representam aqueles que são obesos:



Podemos, então, estabelecer um threshold e classificar novas amostras

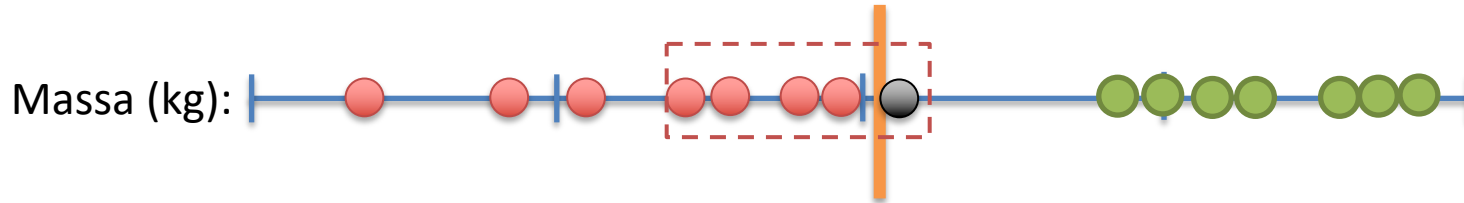


Entretanto, como essa amostra seria classificada?



Como ela está acima do threshold, a classificaríamos como obesa

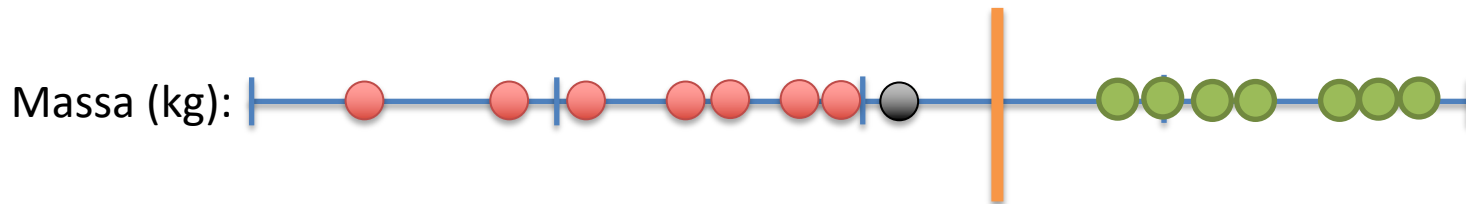
O que não faz sentido, visto que ela é muito mais próxima das amostras da outra classe



Assim, precisamos de uma maneira mais inteligente de definir o threshold.

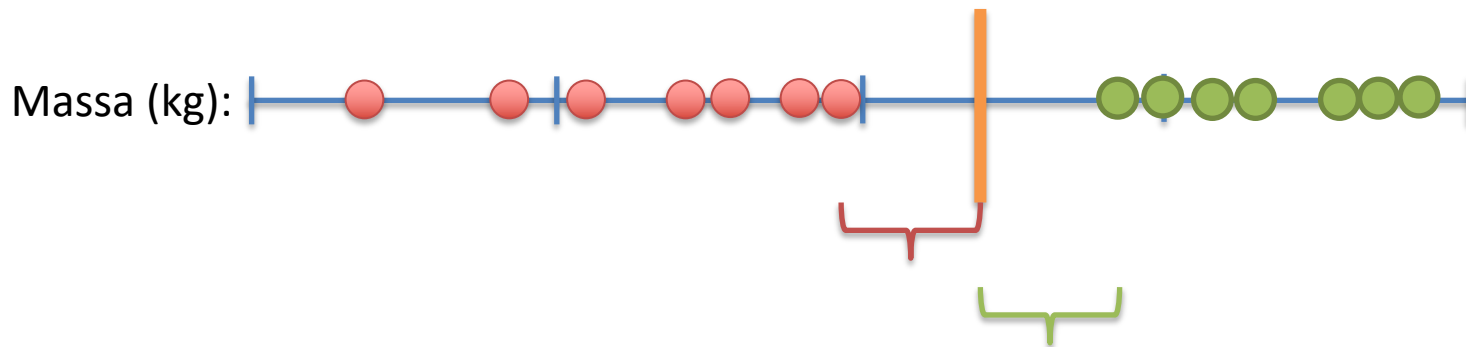
Sugestões?

Voltando aos dados originais, podemos escolher os pontos que estão na borda de cada grupo e usar o ponto médio entre eles como threshold:



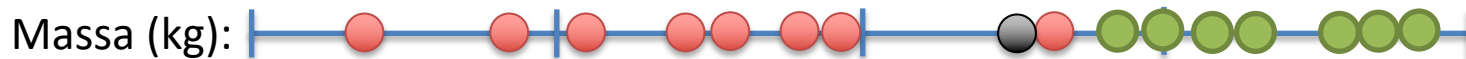
Agora, a nova amostra será posicionada a esquerda do threshold e será classificada como não obesa

O que fizemos ao escolher esse ponto como threshold foi definir uma **margem**, ou seja, a menor distância entre pontos de distintas classes e o threshold.



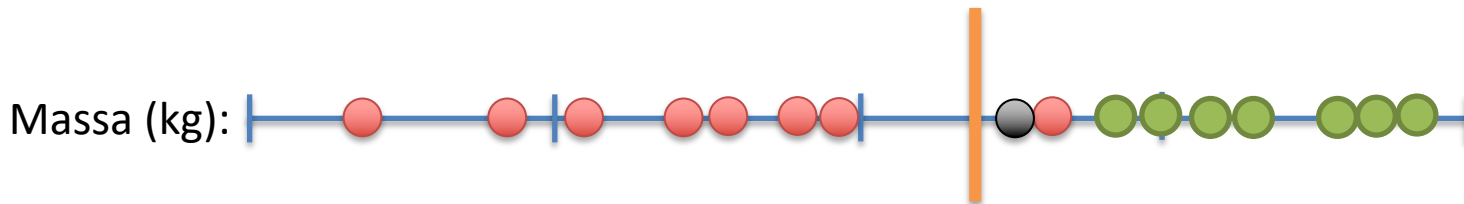
Quando usamos um threshold que nos fornece a maior margem para fazer as classificações estamos usando um ***Maximal Margin Classifier***.

Maximal Margin Classifier podem parecer a solução para nossos problemas, mas e se o conjunto de treino for assim?



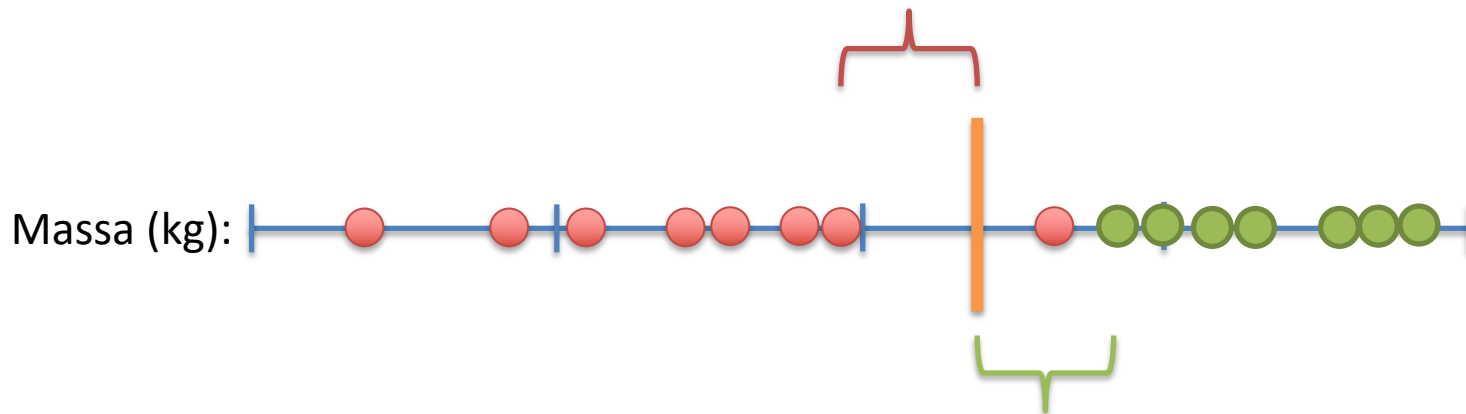
Dessa maneira, novas amostras seriam incorretamente classificadas. Isso torna ***Maximal Margin Classifier*** muito sensível a outliers.

Para resolver esse problema, precisamos estabelecer um threshold que não seja sensível a outliers e, para isso, devemos **permitir erros de classificação**.

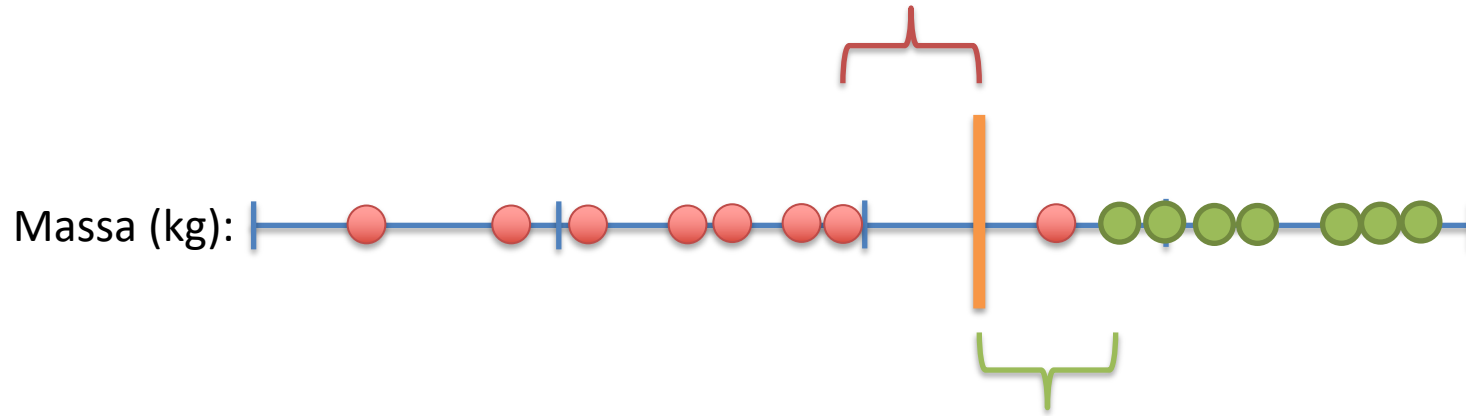


Se, por exemplo, usarmos o threshold acima, iremos errar um ponto (em vermelho), mas permitiria melhores predições.

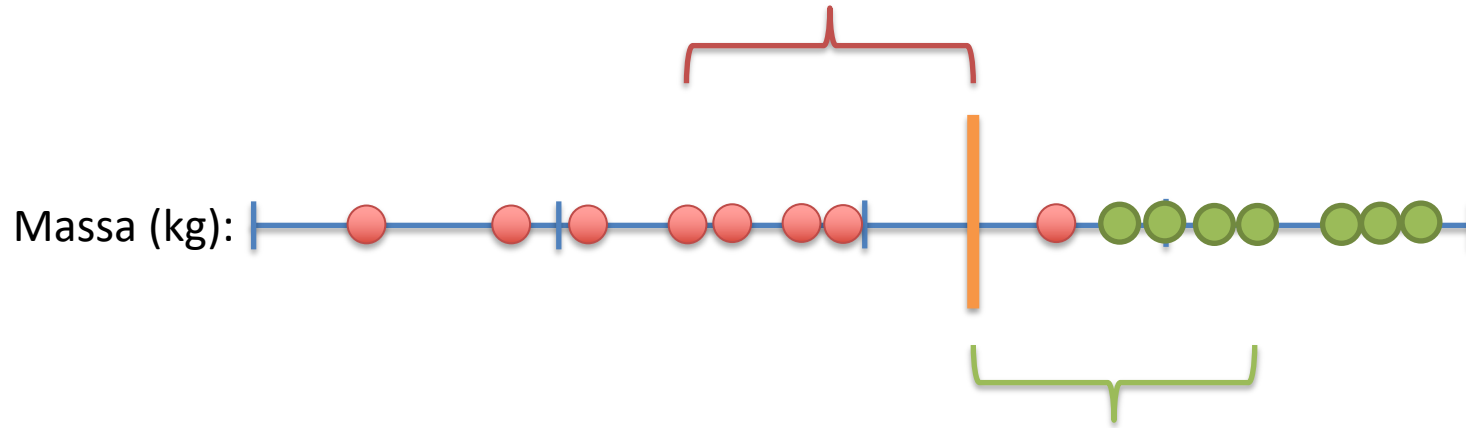
Quando permitimos erros de classificação, a distância entre os pontos e o threshold é chamado de ***Soft Margin***.



Agora a questão é: como eu sei que essa margem...

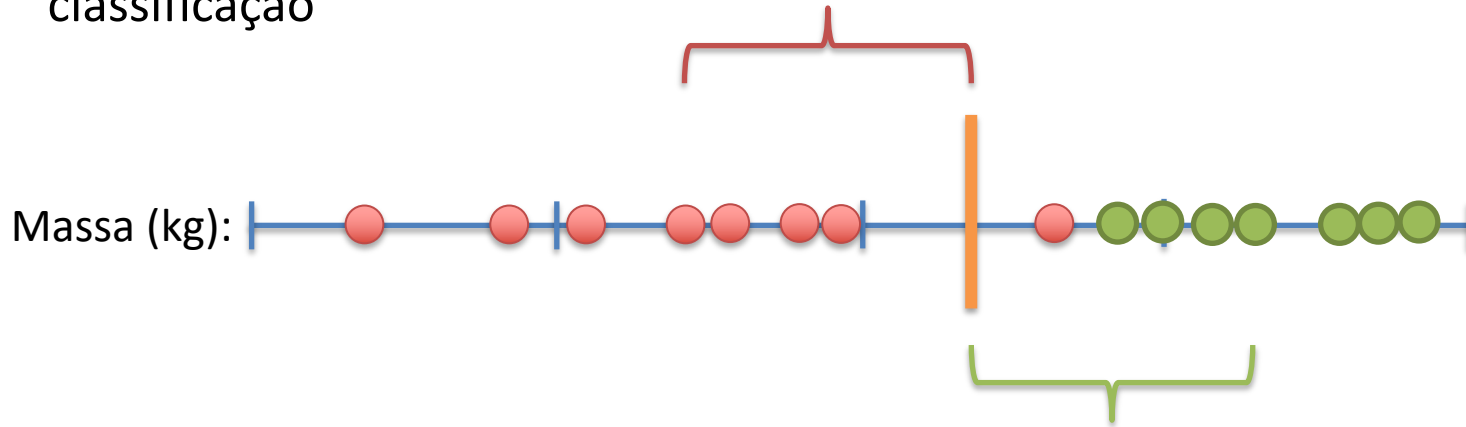


É melhor que essa?

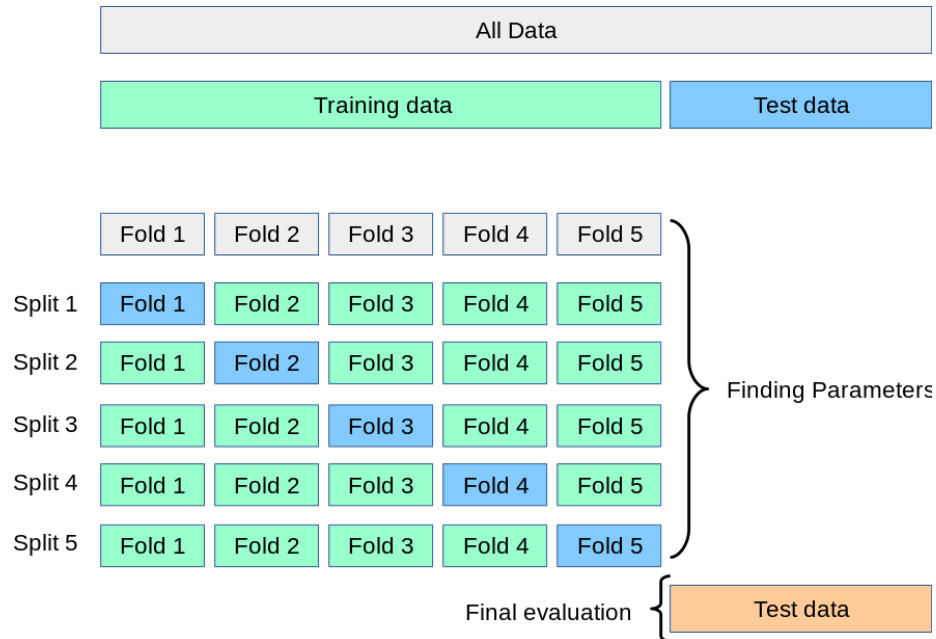


Ou melhor que qualquer outra?

Podemos usar **Cross Validation** para determinar quantos erros de classificação iremos permitir dentro da Soft Margin para obter a melhor classificação

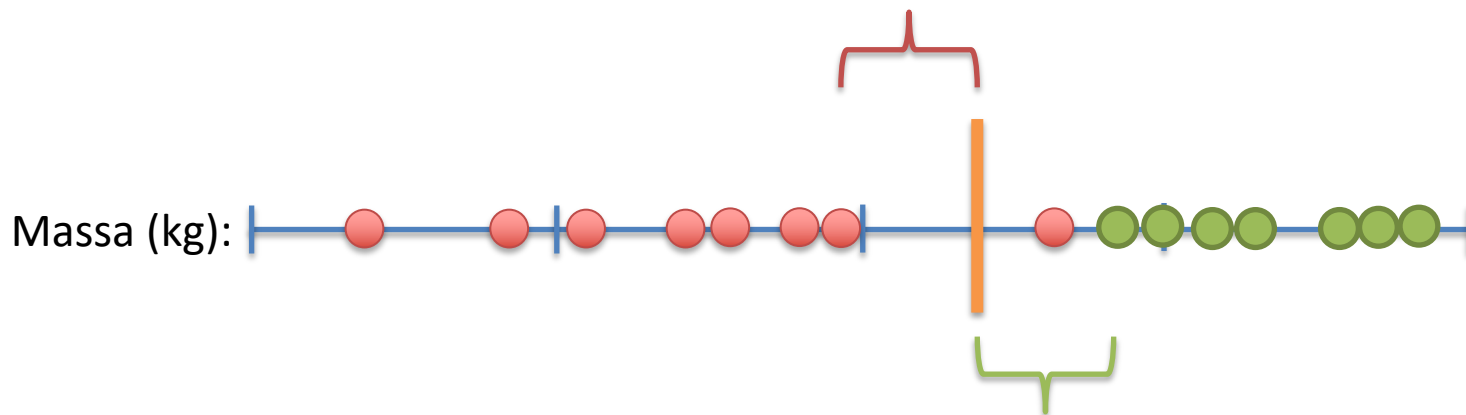


Cross Validation é o processo de dividir um conjunto de treino em N conjuntos de treino e validação e escolher aquele que retorna a menor quantidade de erros de classificação

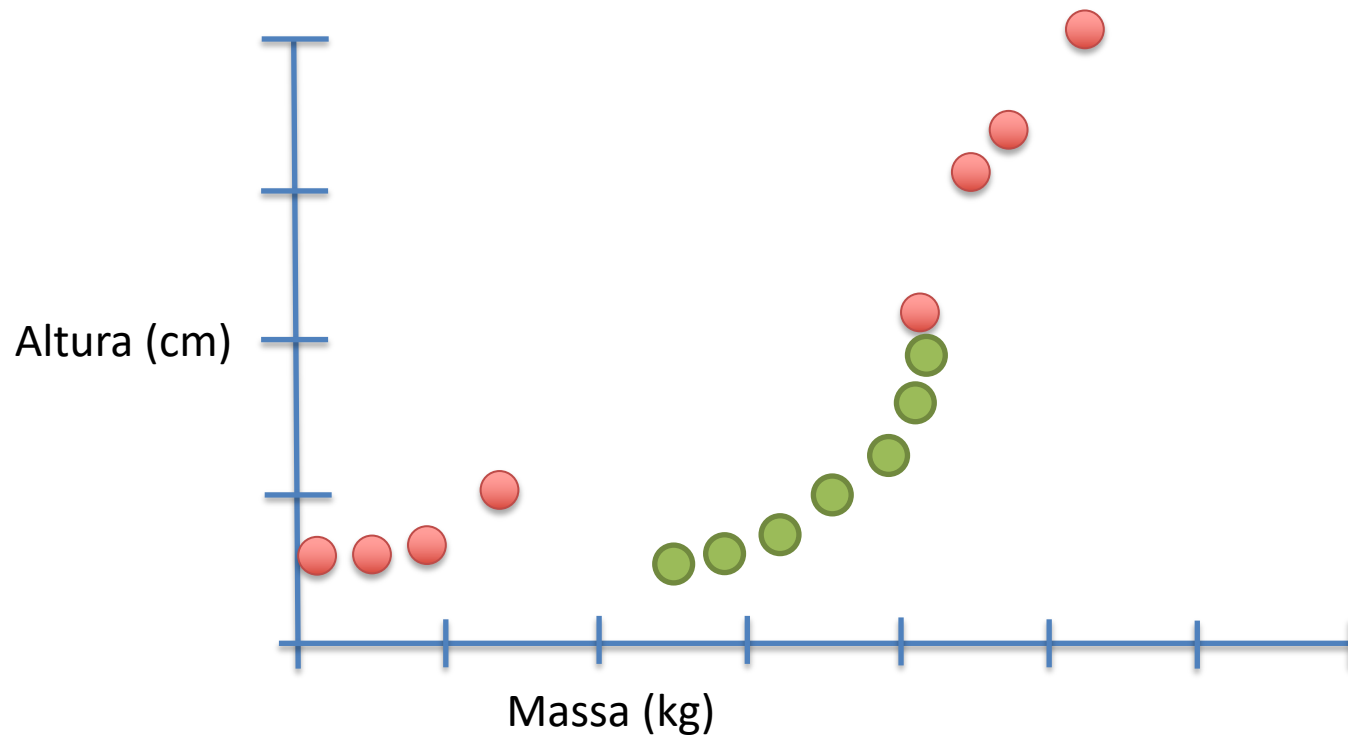


Assim, quando usamos Soft Margin para determinar a localização de um threshold, estamos usando um **Support Vector Classifier** para classificar observações.

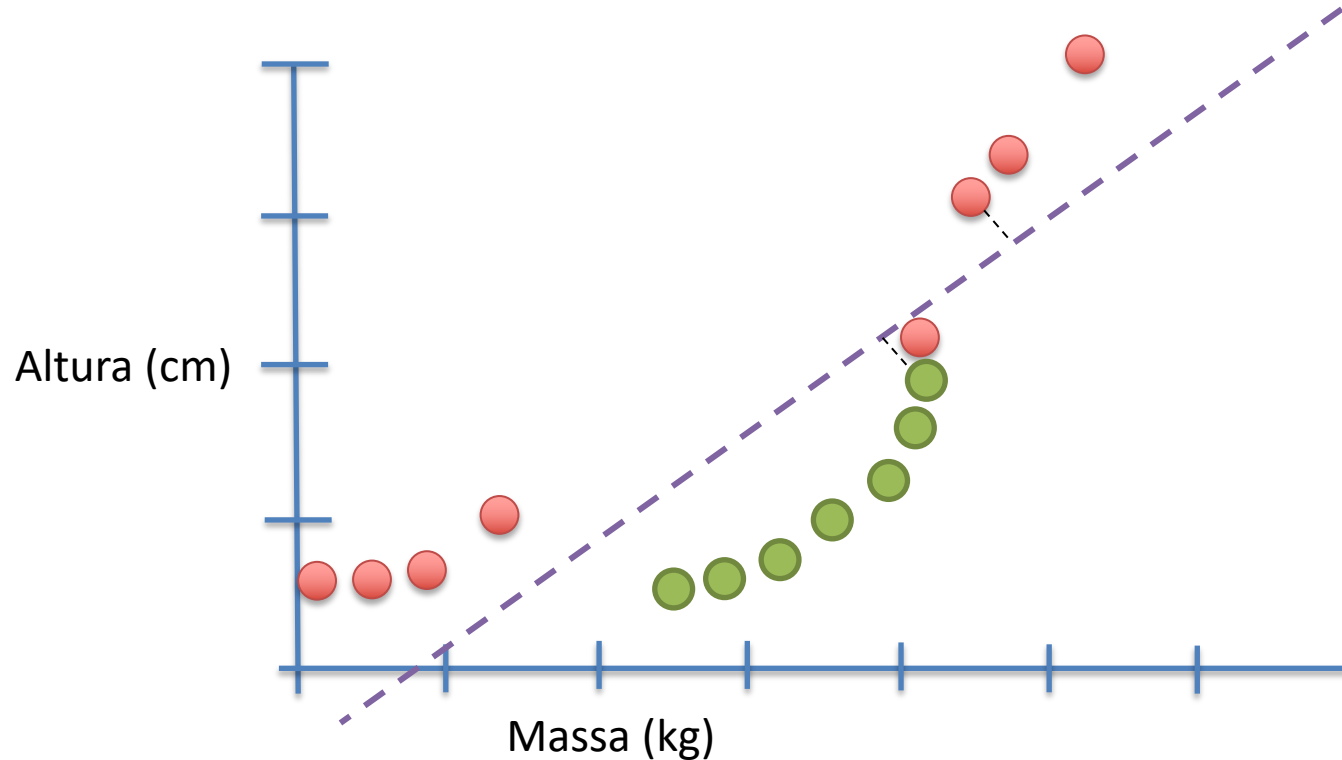
Esse nome vem do fato que observações na borda e entre a Soft Margin são denominadas Support Vector.



Agora, imagine que nossos dados possuem esse comportamento:

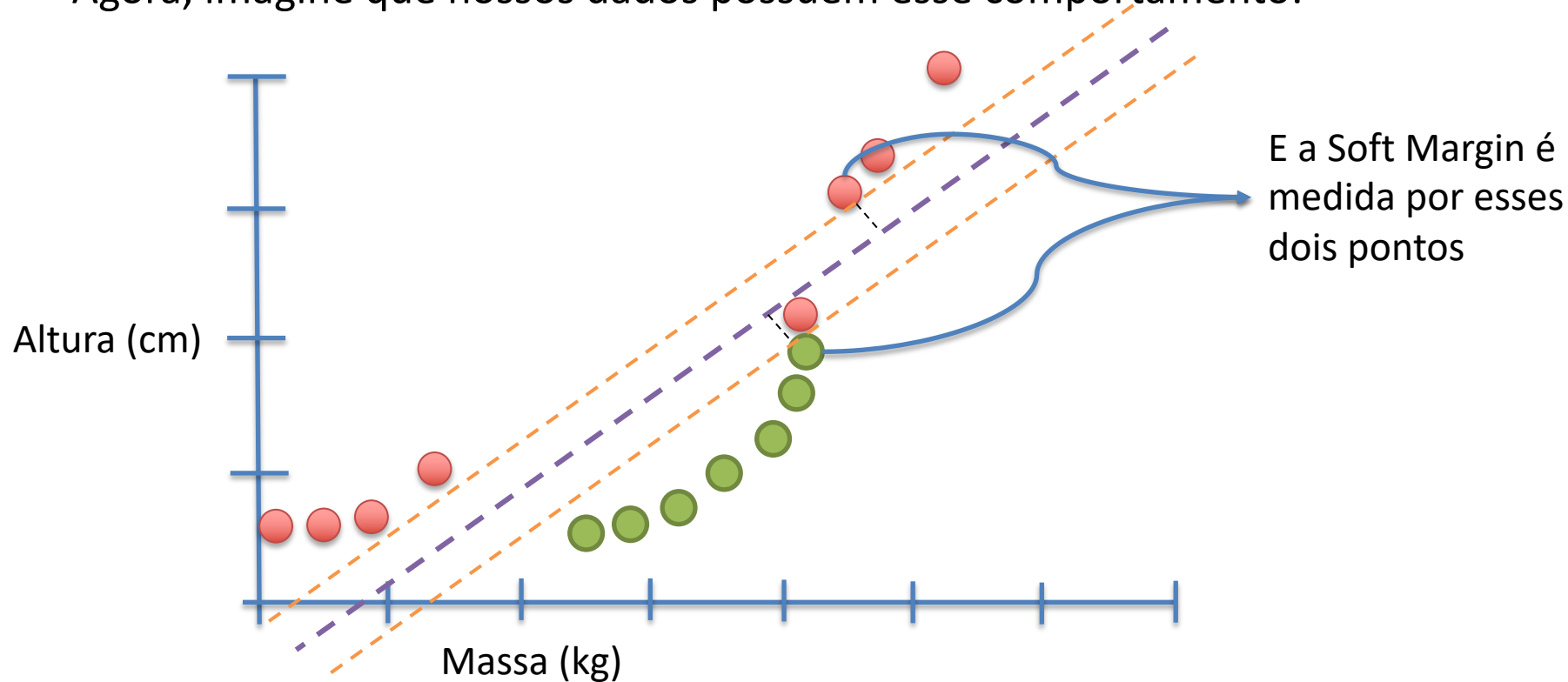


Agora, imagine que nossos dados possuem esse comportamento:

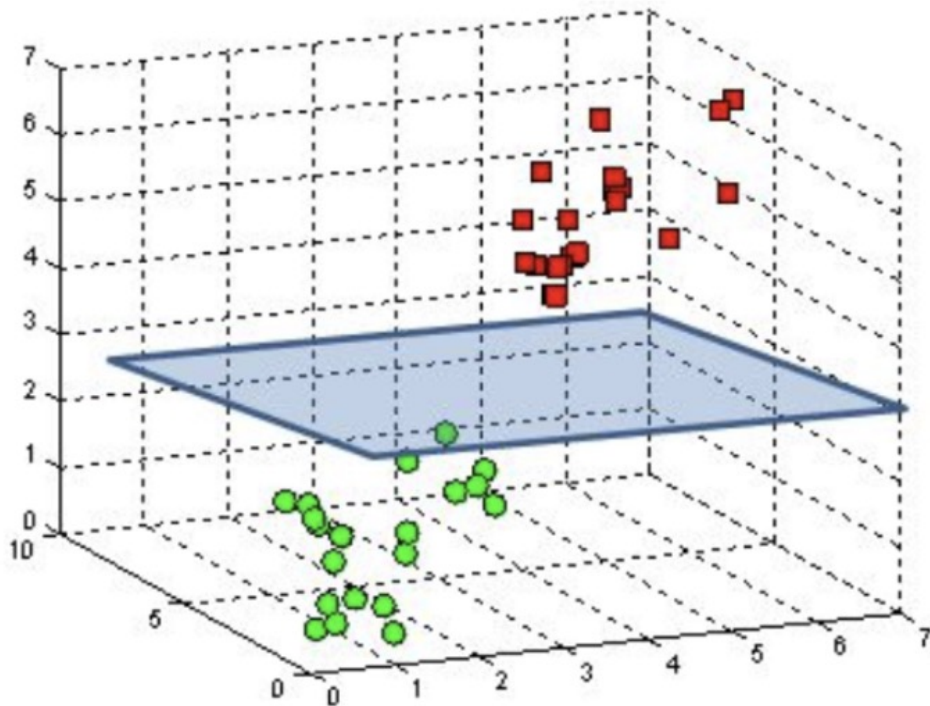


Nesse caso, o
Support Vector
Classifier é uma
linha

Agora, imagine que nossos dados possuem esse comportamento:



Quando os dados possuem 3 dimensões, temos plano como Support Vector Classifier



Para 4 ou mais dimensões, temos um hiperplano (e não tem como representar isso graficamente).

Support Vector Classifiers podem ser interessantes pois eles lidam bem com outliers e, por permitirem erros de classificações na etapa de treino, podem lidar com amostras mais difíceis na etapa de teste.

Entretanto, e se esse for nosso conjunto de treino?

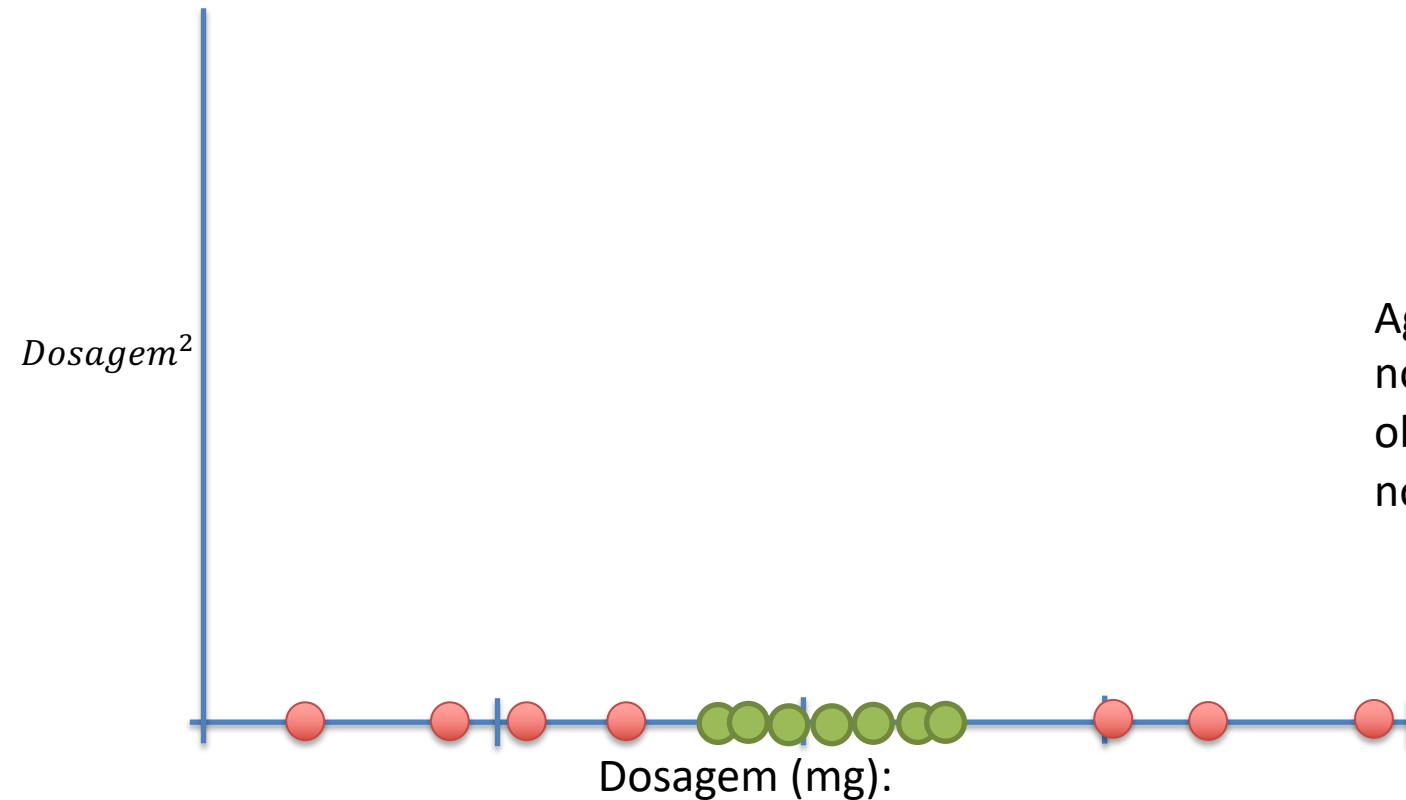


Nesse contexto é que entram Support Vector Machines

Vamos começar entendendo a principal ideia por trás do SVM. Para isso, adicionamos um eixo y para poder desenhar um gráfico:

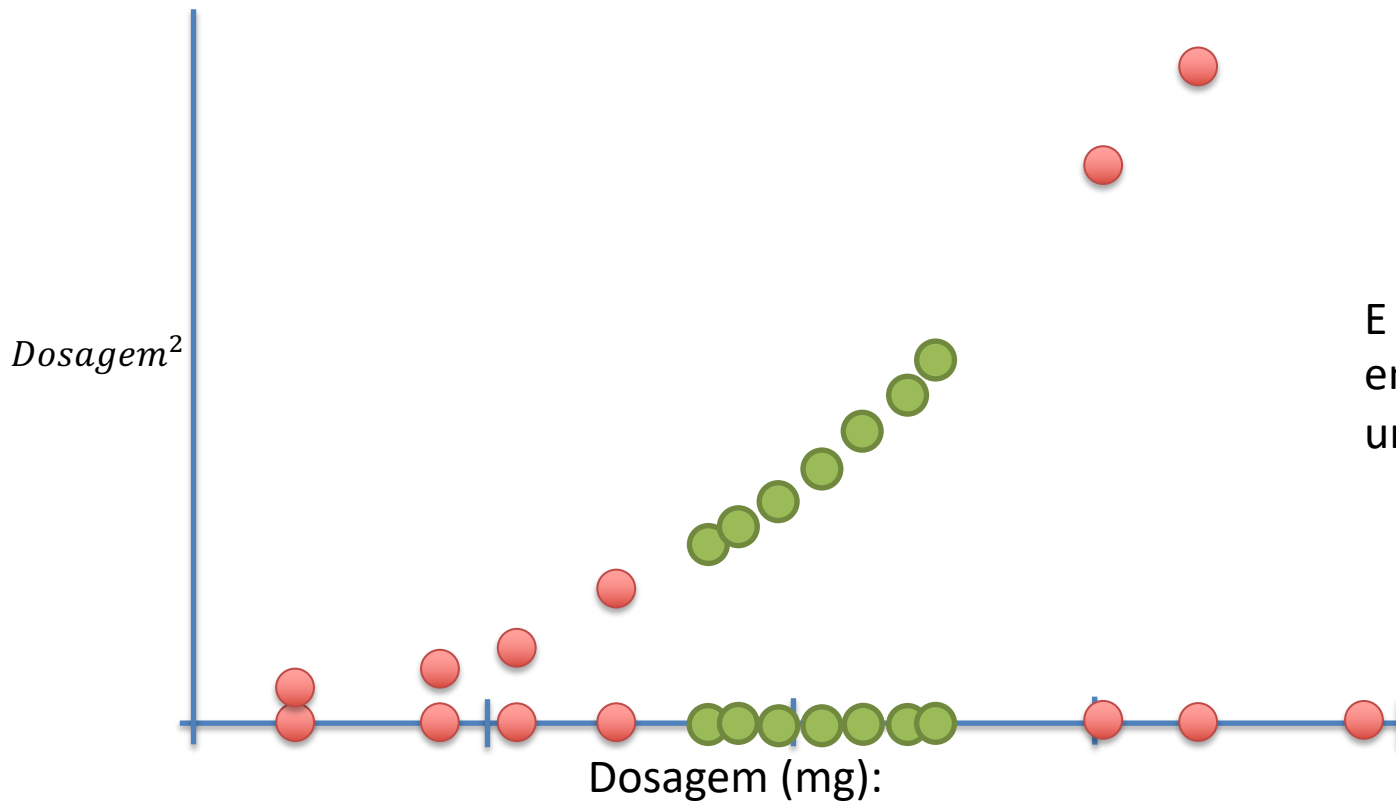


Vamos começar entendendo a principal ideia por trás do SVM. Para isso, adicionamos um eixo y para poder desenhar um gráfico:



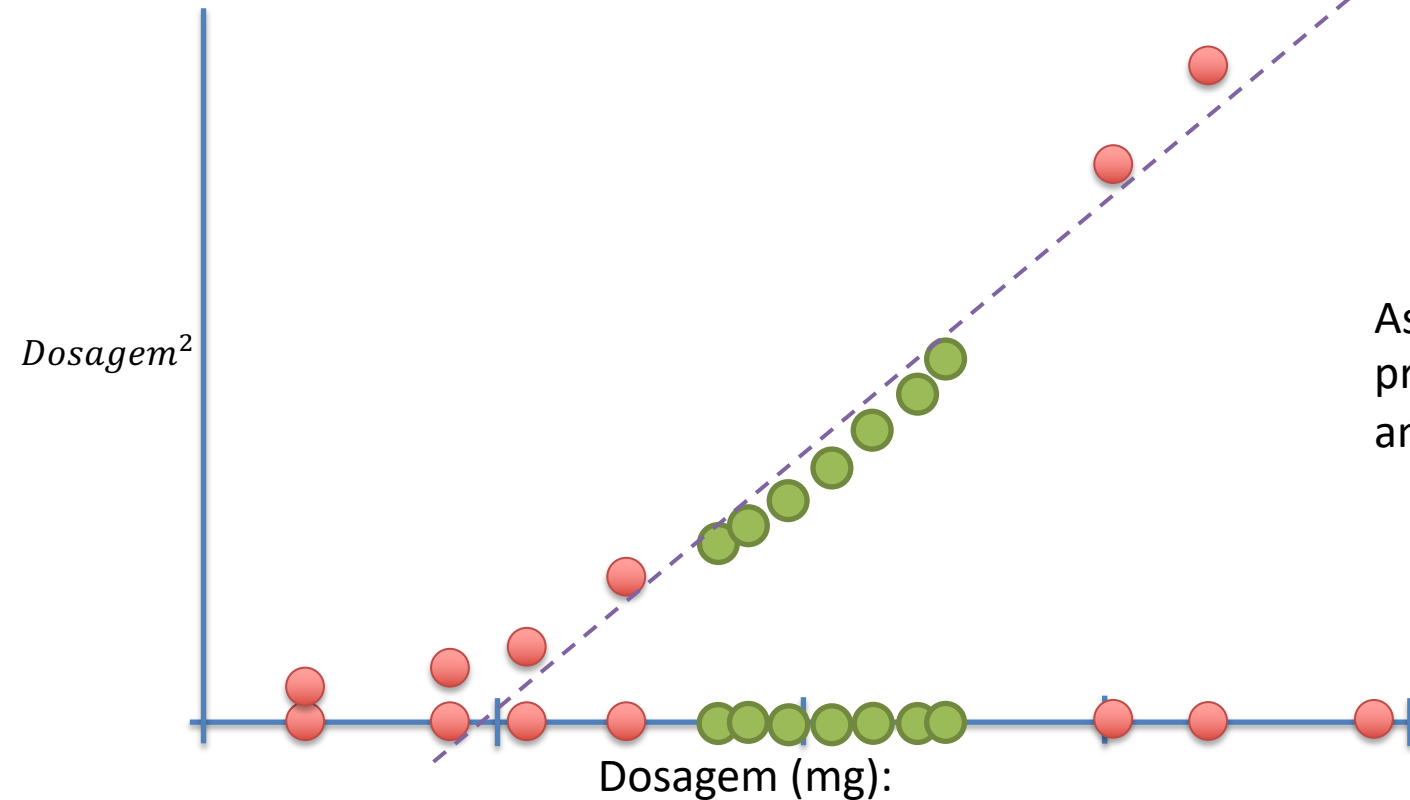
Agora, elevamos cada valor no eixo x ao quadrado para obter seu correspondente no eixo y

Vamos começar entendendo a principal ideia por trás do SVM. Para isso, adicionamos um eixo y para poder desenhar um gráfico:



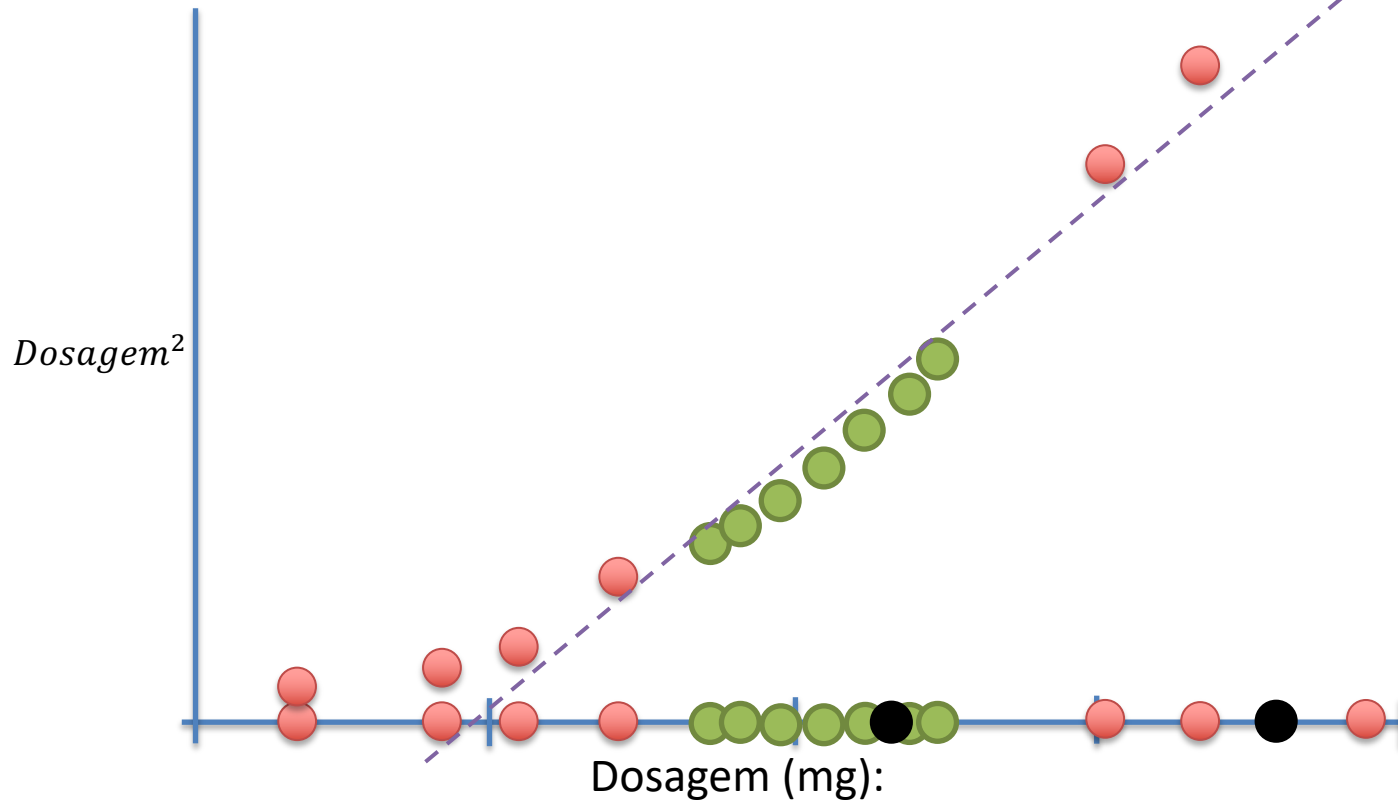
E agora que os dados estão em 2D, podemos desenhar um SVC

Vamos começar entendendo a principal ideia por trás do SVM. Para isso, adicionamos um eixo y para poder desenhar um gráfico:



Assim, é possível fazer previsões de novas amostras. Por exemplo:

Vamos começar entendendo a principal ideia por trás do SVM. Para isso, adicionamos um eixo y para poder desenhar um gráfico:



Assim, podemos listar os passos que compõem a principal ideia por trás do SVM:

- 1. Comece com os dados numa dimensão relativamente baixa***
- 2. Mova os dados para uma dimensão superior***
- 3. Encontre um SVC que separe os dados em grupos distintos***

A questão que permanece, entretanto, é: como decidimos a maneira que vamos transformar os dados?

O SVM usa as chamadas funções Kernel para sistematicamente encontrar SVCs em dimensões maiores.

Aqui, vou tratar dos dois principais Kernels: polinomial e RBF (Radial Basis Function)

O Kernel polinomial é definido da seguinte maneira:

$$k(x, y) = (x^T y + c)^d, \text{ em que:}$$

x e y são vetores de características que representam amostras de treino

c é um parâmetro de regularização, que diz ao SVM o quanto você quer evitar errar a Classificação de cada exemplo de treinamento. Grandes valores de c geram uma margem menor. Já menores valores de c geram uma margem maior, mesmo o hiperplano errando a classificação de mais pontos.

d é o grau do polinômio que vamos usar

Os valores de c e d são obtidos através de *Cross Validation*. (para efeitos de facilidade de cálculo, vou usar $x^T = x$)

Vamos supor que encontramos os seguintes os valores de c e d : $\frac{1}{2}$ e 2 .

Assim, temos a seguinte equação:

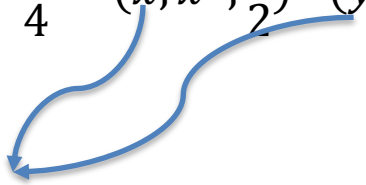
$$(x \cdot y + \frac{1}{2})^2 = \left(x \cdot y + \frac{1}{2}\right) \left(x \cdot y + \frac{1}{2}\right) = \left(x^2 y^2 + \frac{1}{2}xy + \frac{1}{2}xy + \frac{1}{4}\right) = xy + x^2 y^2 + \frac{1}{4}$$

$$\left(x \cdot y + \frac{1}{2}\right)^2 = \left(x \cdot y + \frac{1}{2}\right)\left(x \cdot y + \frac{1}{2}\right) = \left(x^2y^2 + \frac{1}{2}xy + \frac{1}{2}xy + \frac{1}{4}\right) = xy + x^2y^2 + \frac{1}{4}$$

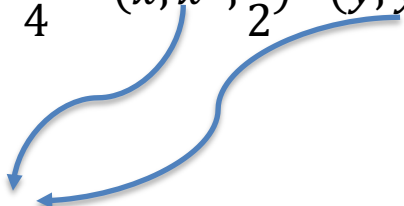
Este polinômio, por fim, é igual a esse ***dot product***:

$$xy + x^2y^2 + \frac{1}{4} = \left(x, x^2, \frac{1}{2}\right) \cdot \left(y, y^2, \frac{1}{2}\right)$$

Esse dot product fornece as coordenadas dos dados numa alta dimensão.

$$xy + x^2y^2 + \frac{1}{4} = (x, x^2, \frac{1}{2}) \cdot (y, y^2, \frac{1}{2})$$


Representam as
coordenadas do eixo
x de duas amostras

$$xy + x^2y^2 + \frac{1}{4} = (x, x^2, \frac{1}{2}) \cdot (y, y^2, \frac{1}{2})$$


Representam as
coordenadas do eixo
y de duas amostras

$\frac{1}{2}$ e $\frac{1}{2}$ representam as
coordenadas no eixo z, mas
como estamos usando um
polinômio de grau 2,
podemos ignorá-las.

Se quisermos saber a relação entre duas amostras numa alta dimensão (definida pelo kernel da equação), basta substituir seus valores no lugar de x e y .

Observe que, ao fazer isso, os dados **não são** de fato transformados para uma outra dimensão. Essa equação que nos permite encontrar as relações entre amostras em outras dimensões sem de fato transformar os dados é chamado de kernel trick.

O Kernel *Radial Basis Function* (RBF) é definido da seguinte maneira:

$$e^{-\gamma \|x_i - x_j\|^2}, \text{ em que:}$$

e representa o número de Euler = 2,71828

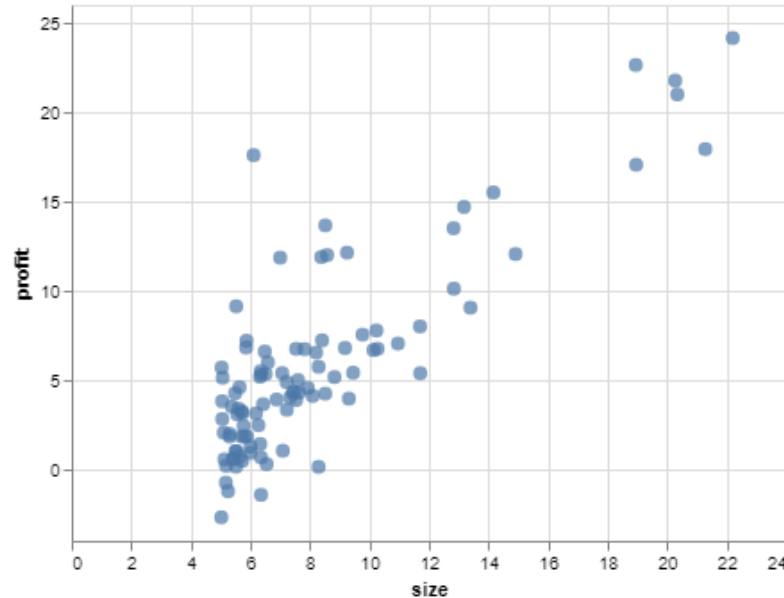
No jupyter notebook eu dou mais detalhes do funcionamento da função.

Implementação

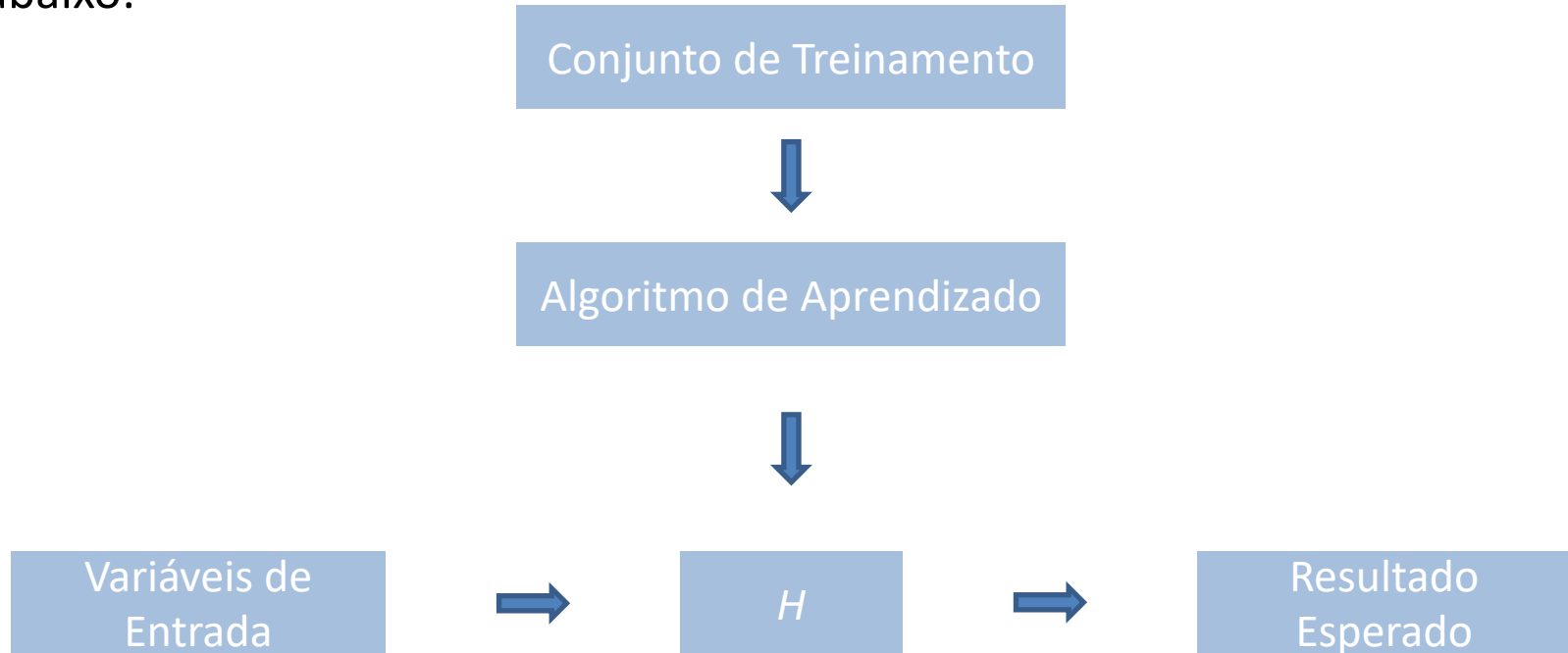


Regressão Linear

Suponha que eu deseje criar um algoritmo de machine learning cujo objetivo é estimar se devo ou não abrir uma filial de minha loja num determinado lugar. Para isso, disponho de um conjunto de dados que relaciona lucro e população

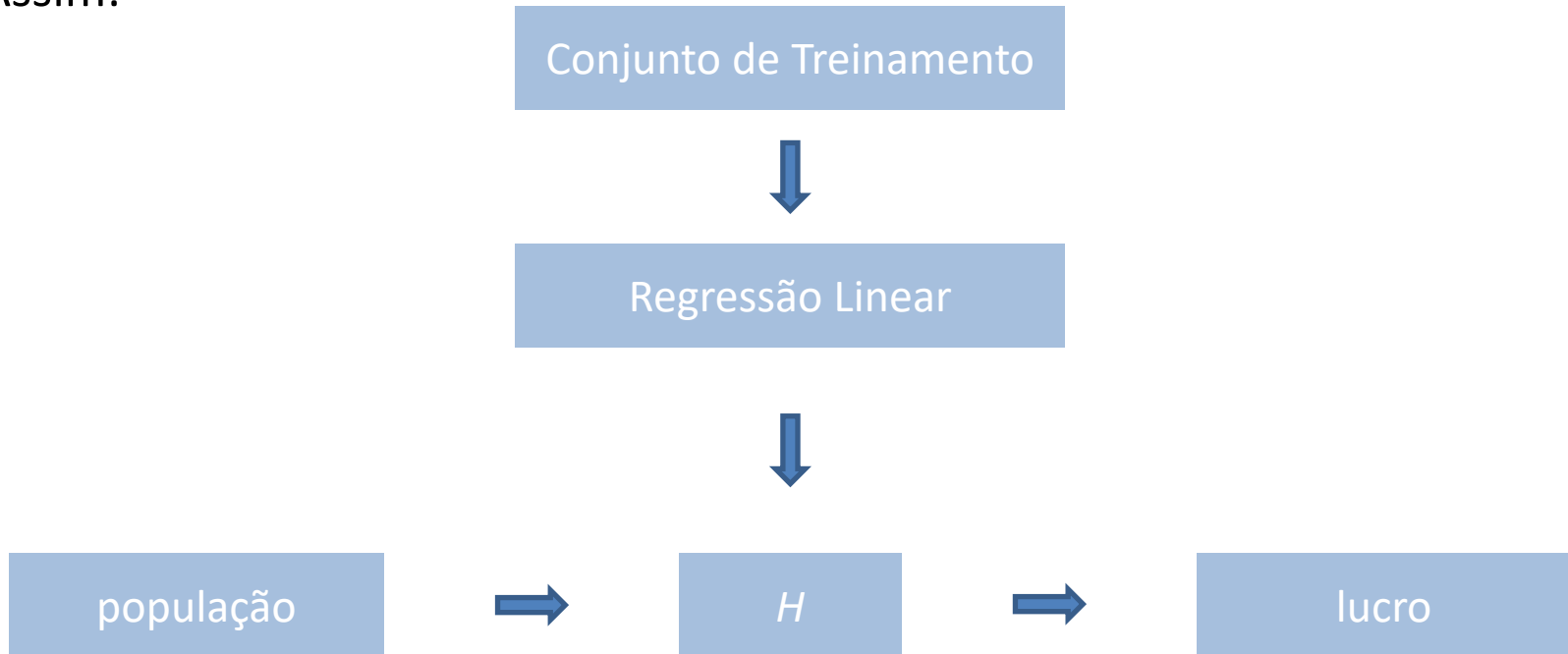


Nosso objetivo é criar um algoritmo que consiga prever qual o lucro que terei numa cidade a partir da quantidade de pessoas que lá vivem. De modo geral, podemos representar esse objetivo a partir do diagrama abaixo:



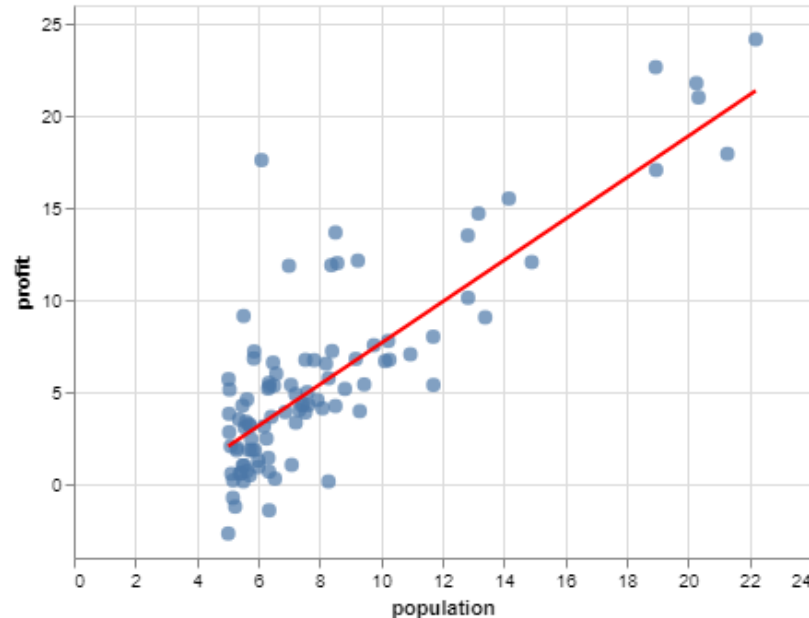
H representa uma hipótese que, a partir de um algoritmo de aprendizado, mapeia as variáveis de entrada para um resultado esperado.

Assim:



Como o próprio nome diz, a Regressão Linear irá ajustar uma reta aos dados a fim de permitir que eu estime o lucro a partir da população.

Desse modo, esperamos como resultado da Regressão Linear, algo do tipo:



Duas perguntas surgem, então:

1. Qual é a **hipótese da Regressão Linear** que me permite ajustar uma reta aos dados?
2. Dada a minha hipótese, como eu sei que a reta obtida é a **melhor possível**?

Vamos começar definindo nossa hipótese.

Se eu ajusto uma reta aos meus dados, nada mais óbvio que recorrer à Álgebra Linear e lembrar qual equação dá origem à uma reta:

- $$h_{\theta}(x) = \theta_0 + \theta_1 x$$

θ_0 representa o coeficiente linear, ou seja, o valor numérico (ponto) por onde a reta passa no eixo das ordenadas (y).

θ_1 , ou coeficiente angular, representa a inclinação da reta em relação ao eixo das abscissas (x).

Vamos analisar alguns exemplos de hipóteses para entender como a reta é criada a partir de diferentes valores de θ_0 e θ_1 :

- $h_{\theta}(x) = 1.5 + 0x$
- $h_{\theta}(x) = 0.5x$
- $h_{\theta}(x) = 1 + 0.5x$

Dos exemplos, entendemos que, para determinar uma reta que se ajuste aos dados, precisamos determinar os valores de θ_0 e θ_1 .

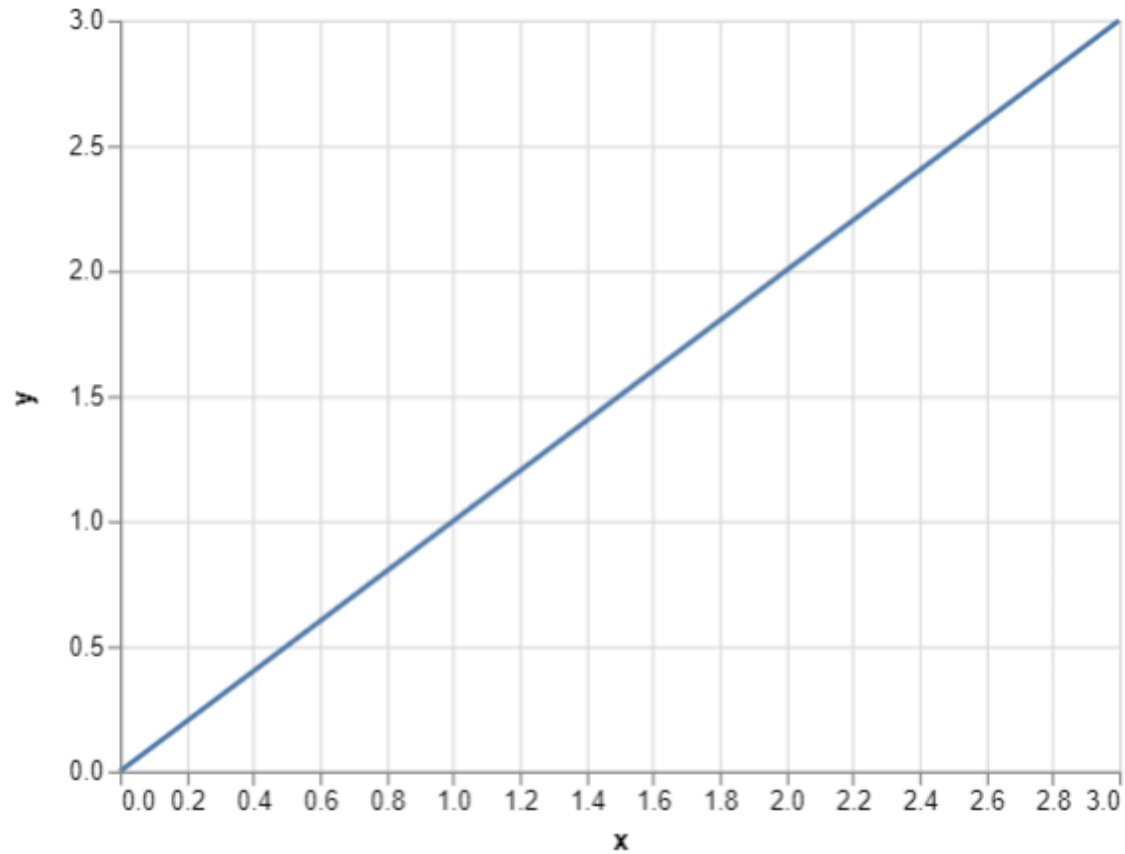
Há várias maneiras de determinarmos esses valores, mas precisamos de uma que nos permita dizer o quão bom são esses valores.

Aqui entra em cena nossa **Função de Custo**

Considere um conjunto fictício cujas respostas para uma determinada entrada de dados seja $y = [0,1,2,3]$.

Considere, ainda, que minha regressão linear ajustada a esses trouxe como resposta $\gamma = [0,1,2,3]$

Diante disso, qual o erro de predição de meu algoritmo?

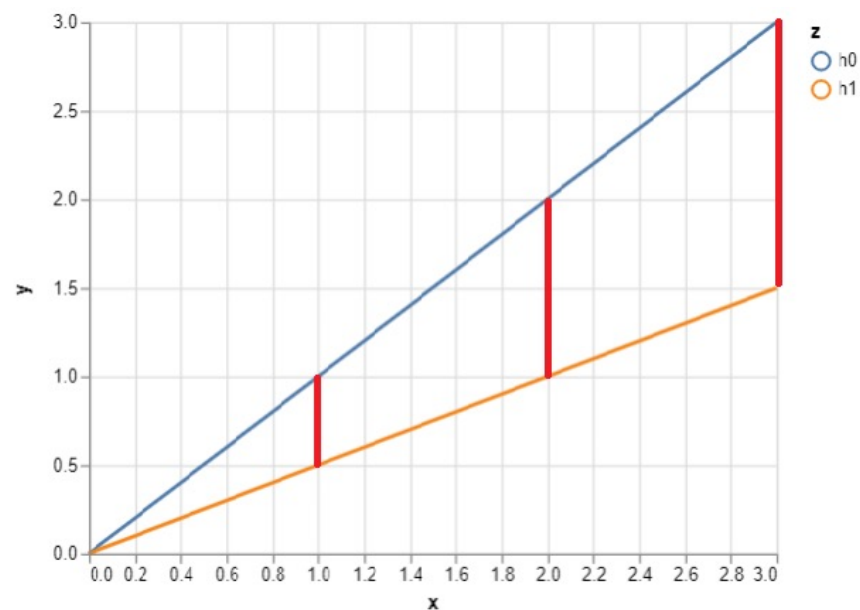
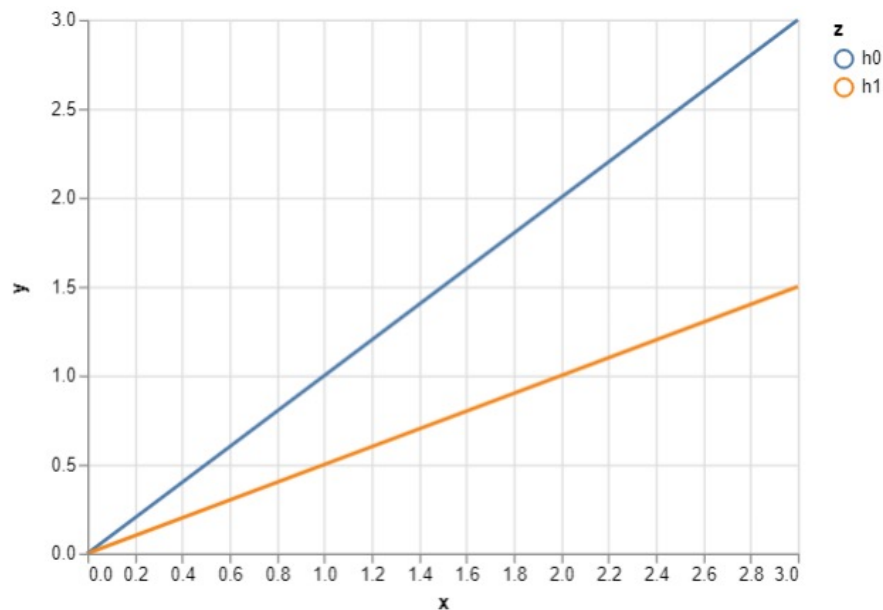


Considere as mesmas respostas $y = [0,1,2,3]$.

Considere, agora, que minha regressão linear ajustada a esses trouxe como resposta $\gamma = [0,0.5,1,1.5]$

Qual o erro de predição de meu algoritmo agora?

O erro pode ser representado pela diferença entre o que foi predito e o que de fato é



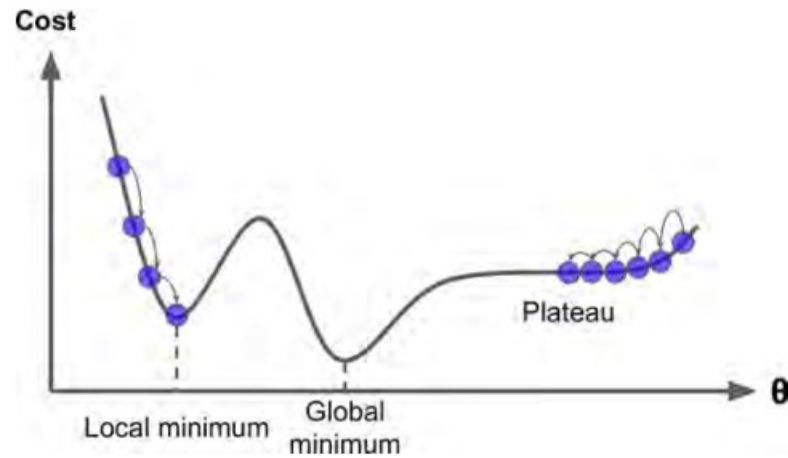
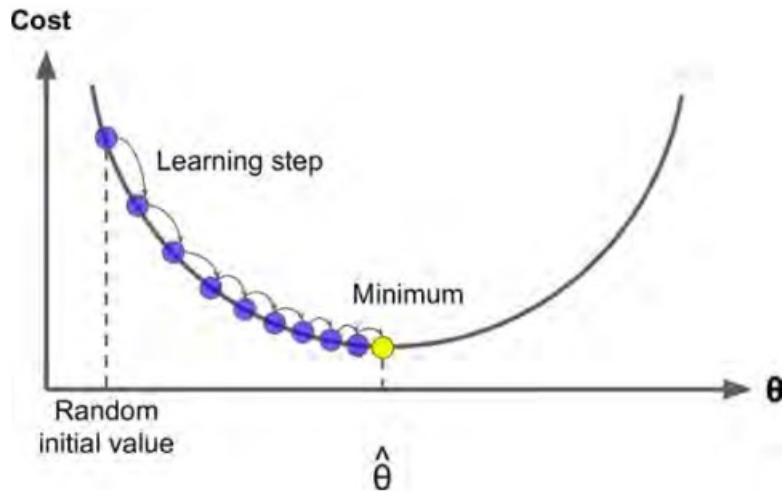
Assim, conseguimos definir nossa função de custo como sendo a diferença entre o valor predito e o valor real.

Matematicamente:
$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$$

Em que:

- $J(\theta_0, \theta_1)$: função de custo aplicada aos parâmetros θ_0 e θ_1
- m é a quantidade de exemplos de treinamento
- $h_{\theta}(x^i)$ é o valor predito pela Regressão Linear
- y^i é o valor real
- Elevamos ao quadrado para evitar valores negativos
- Tiramos a média para facilitar a derivada ao calcular o Gradiente Descendente

Por fim, para determinar os valores de θ_0 e θ_1 , usaremos o Gradiente Descendente, cuja ideia é, iterativamente, alterar os parâmetros θ_0 e θ_1 até chegar num mínimo, que representará um bom ajuste da reta aos dados.



Para eu caminhar por minha função até atingir um mínimo, eu uso **derivadas**. Então, eu aplico o gradiente descendente à minha função de custo, obtendo a seguinte equação:

$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

α representa o meu **learning rate**, que simboliza o tamanho do passo que irei dar para caminhar até o mínimo da minha função.

- Se o passo for curto demais, levarei mais iterações para atingir o mínimo;
- Se for longo demais, minha função pode não convergir, ou mesmo divergir.

Posso reescrever a seguinte equação

- $$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

Como:

- $$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$$

Resolvendo a derivada para θ_0 e θ_1 , temos:

- $$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 x^i - y^i)$$

- $$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 x^i - y^i) x^i$$

Para mais detalhes sobre o processo derivativo, acesse esse [link](#)

Resumidamente, para aplicar Regressão Linear a um conjunto de dados, eu faço:

1. Início θ_0, θ_1 com algum valor aleatório (geralmente 0)
2. Calculo o gradiente aplicado à função de custo:
 1. $\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$
3. Iterativamente, repito o processo até minha função de custo chegar a um valor mínimo ou até atingir a quantidade de iterações estabelecida

Implementação

Regressão Linear Multivariada

Até então, usamos apenas a feature população para estimar o lucro e decidir se vamos ou não abrir uma filial de nossa loja nessa cidade.

Na vida real, muitos outros fatores (features) influenciam essa tomada de decisão: renda per capita, localização, logística, etc.

Assim, estamos falando agora de **várias variáveis que irão compor nossa hipótese**

Desse modo, tratamos θ e x como vetores de $n + 1$ posições, em que n é a quantidade de características.

- $$h_{\theta}(x) = \theta_0 x_0 + \theta_1 x_1 \dots \theta_n x_n$$

Por conveniência, adotamos $x_0 = 1$, assim, podemos multiplicar esses dois vetores para obtermos nossa hipótese

Vejamos o exemplo do dataset consumo_cerveja

Logo, o gradiente descendente aplicado à Regressão linear multivariada é dado por:

$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0 \dots \theta_n)$$

De modo detalhado, temos:

$$\theta_j = \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^i) - y^i) x_j^i$$

Implementação

Equação Normal

Até agora, para encontrarmos os valores de θ , usamos gradiente descendente. Entretanto, existe outro método que, analiticamente, determina os valores de θ . É denominado Equação Normal

Seja X a matriz de valores das features e γ o vetor dos valores do atributo meta, podemos calcular os valores de θ seguindo a equação:

- $$\theta = (X^t X)^{-1} X^t \gamma$$

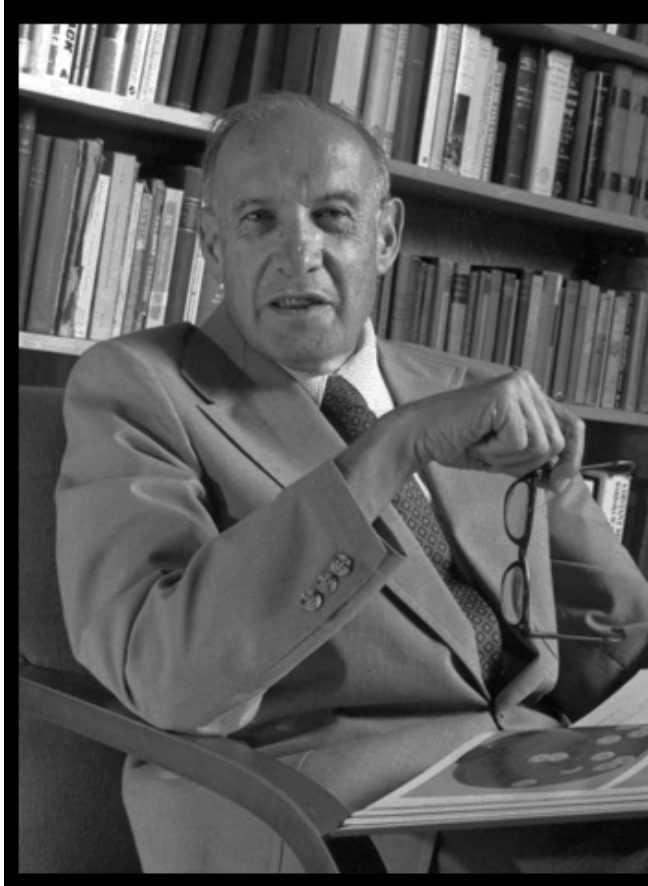
A explicação matemática de como chegar nessa função e do porquê ela funciona está além do escopo de nossa aula, mas nesse [link](#), vocês podem encontrar essa resposta

Mas quando usar Gradiente Descendente e quando usar equação normal?
Aqui comparamos as duas abordagens:

- Gradiente Descendente:
 - Necessário escolher α
 - Necessário muitas iterações
 - Funciona bem quando n é grande
- Equação Normal:
 - Sem necessidade de escolher α
 - Sem necessidade de iteração
 - Cálculos de ordem O^3 ([complexidade de algoritmos](#))
 - Lento se n é muito grande

Implementação

Métricas de Avaliação



You can't
manage what
you can't
measure

-Peter Drucker

Precisamos de maneiras de mensurar a qualidade de predição de nosso algoritmo. Em classificação, as métricas mais comuns, e as mais simples, são **Acurácia**, **Precision**, **Recall** e **F1 Score**. Elas são obtidas a partir da [Matriz de Confusão](#), apresentada abaixo:

		True Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

A acurácia é dada pela seguinte função:

$$acc = \frac{tp + tn}{(tp + tn + fp + fn)}$$

Precision (acurácia das predições positivas) pode ser calculado da seguinte maneira:

$$P = \frac{tp}{(tp + fp)}$$

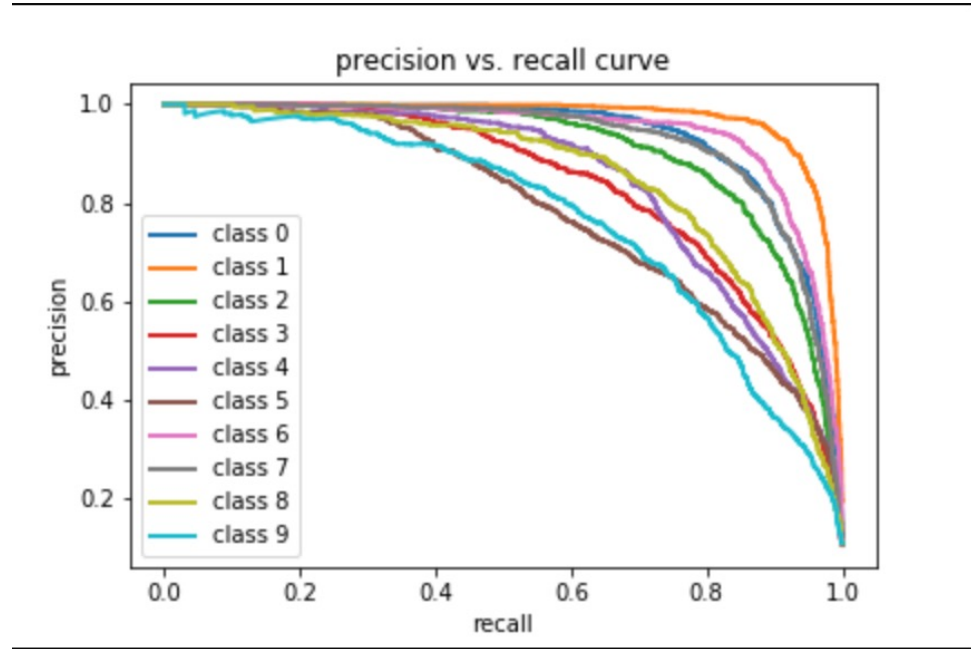
Já o Recall (taxa de amostras positivas corretamente encontradas pelo classificador) é calculado da seguinte forma:

$$R = \frac{tp}{(tp + fn)}$$

Precision e **Recall** tendem a ser inversamente proporcionais, como apresentado no gráfico ao lado.

Para minimizar esse problema, existe a métrica **F1-Score**, que leva em consideração tanto Precision quanto Recall.

$$F1 = \frac{2PR}{P + R}$$



Para ampliar o conhecimento:

- pesquisem sobre essas duas outras métricas: ROC e AUC
- Ponto de partida: [link](#)

Exercícios



Obrigado!

profdheny.fernandes@fiap.com.br

 /dhenyfernandes

FIAP MBA⁺

Copyright © 2022 | Professor Dheny R. Fernandes

Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento, é expressamente proibido sem consentimento formal, por escrito, do professor/autor.

FIAP